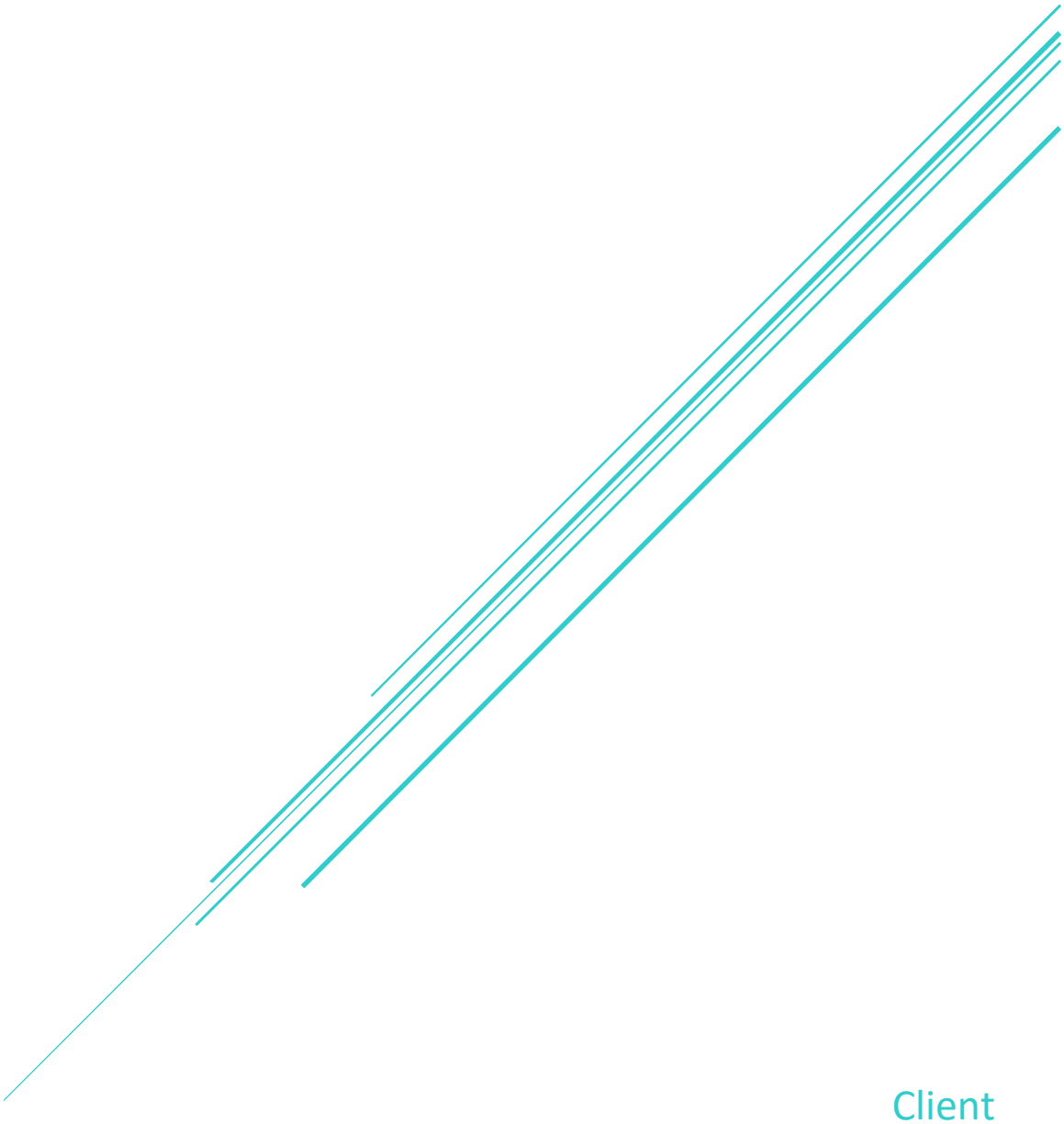
DESIGN AND IMPLEMENTATION OF AN MLOPS-ORIENTED PIPELINE FOR CMOS VOC SENSOR DATA IN EARLY VETERINARY DISEASE DETECTION

Adityanand Singh, Faezeh Kianimoravej, Sepideh Qorbani



Client
Eyuel Ayele

Contents

1. Abstract

This project presents the design and implementation of an end-to-end machine learning pipeline for CMOS VOC gas-sensing data, with an emphasis on reproducibility, modularity, and deployment readiness. The system addresses the main challenges of time-series gas classification, including run-to-run variability, non-stationary sensor behavior, variable sequence length, and limited data volume. To handle these issues, the pipeline transforms raw signals through run-level preprocessing, windowing, and feature engineering before training and evaluating models under leakage-safe, group-aware validation.

A hierarchical two-stage modeling strategy is adopted. Stage 1 performs mixture detection, where Random Forest is selected as the final model based on the best run-aware cross-validation performance, achieving 96.1% accuracy and an F1-score of 0.96. Stage 2 focuses on gas-type identification within the single-gas subset, where tuned Logistic Regression is selected as the final model, achieving 79% accuracy and an F1-score of 0.62. This stage reflects the difficulty of distinguishing gases with overlapping response patterns, especially Toluene and 2-butanone. The full workflow is orchestrated using Airflow and versioned through DVC, while MLflow tracks experiments and Docker supports consistent execution. The system also includes FastAPI-based serving and GitLab CI/CD integration for validation and deployment.

Overall, the project demonstrates that effective gas-sensing machine learning depends not only on model choice, but also on robust preprocessing, leakage-aware evaluation, and a well-structured MLOps architecture that supports traceability, reproducibility, and practical deployment.

2. Introduction

Gas sensing plays a critical role in environmental monitoring and livestock management, where the detection of volatile organic compounds (VOCs) can provide valuable insights into animal health and surrounding conditions. Traditional gas sensing approaches often rely on threshold-based detection using single sensors, which limits their ability to distinguish between gases with similar chemical properties and to operate reliably in complex environments.

Modern sensing systems address these limitations by leveraging sensor arrays and data-driven techniques to capture distinctive response patterns, often referred to as gas fingerprints. However, these systems introduce new challenges, including high-dimensional data, variability across measurements, and strong temporal dependencies. As a result, designing robust machine learning pipelines for gas sensing requires careful consideration of both data characteristics and system-level constraints.

This project focuses on developing an end-to-end data and machine learning pipeline for analyzing time-series sensor data in a gas sensing context. The goal is to design a system capable of transforming raw sensor signals into meaningful predictions, while ensuring reproducibility, scalability, and robustness. The work is conducted in collaboration with a real-world stakeholder, emphasizing practical applicability and alignment with domain requirements.

To achieve this, a structured approach inspired by the CRISP-DM methodology is followed. The project begins with a thorough analysis of the problem domain and dataset characteristics, followed by the design of a feature-based modeling strategy tailored to temporal signal behavior. The system employs a two-stage hierarchical modeling approach: first classifying single-gas versus mixture scenarios, then estimating probabilities for specific single gases (e.g., Toluene versus 2-butanone). The system is further extended with MLOps components, including orchestration, experiment tracking, continuous integration, and cloud-based deployment, resulting in a complete and operational pipeline.

The remainder of this report is structured as follows. Section 3 presents the related work and summarizes the literature review conducted for this project [7] on gas sensing systems, sensor data processing, and machine learning-based pattern recognition. Section 4 describes the business and problem understanding, including the problem context, system requirements, and design goals. Section 5 explains the process management approach followed during the project. Section 6 provides the data understanding phase, while Section 7 presents the data preparation and feature engineering strategy.

Section 8 describes the modeling approach, and Section 9 presents the evaluation strategy and results. Section 10 introduces the system design and MLOps architecture, including the main architectural components, pipeline design, and technology choices. Section 11 describes the deployment and implementation of the system. Section 12 discusses the main findings and implications of the results. Section 13 focuses on quality control and reproducibility. Finally, Section 14 outlines the limitations of the current system and directions for future work.

3. Related work

We conducted a literature review [7] to examine prior studies on gas sensor systems and to identify the main methodological ideas already explored for CMOS VOC sensor analysis. The goal was to understand how researchers have addressed the core challenges of these sensors, including noise, drift, cross-sensitivity, run-to-run variability, temporal structure, and limited data availability. The main conclusion is that reliable gas sensing does not depend on a single modeling choice alone, but on the combination of preprocessing, segmentation, feature engineering, validation strategy, and model selection.

Previous work shows that raw gas sensor signals are difficult to use directly because they are affected by scale differences, measurement noise, missing values, and changes across experimental runs. For this reason, preprocessing is consistently treated as a necessary first step. Normalization is used to make signals comparable across sensors and experiments, while smoothing methods are applied to reduce noise without destroying the underlying signal structure. Robust handling of missing values is also important because incomplete measurements can distort downstream analysis and reduce model stability (Literature Review, Section 2, Data Preprocessing & Normalization) [7].

A second major finding is that gas sensing should be treated as a time-series problem. Sensor responses evolve through baseline, exposure, and recovery phases, and these temporal changes often contain more information than any single static value. Related studies therefore favor segmentation methods such as fixed-length windows and sliding windows, which convert variable-length signals into standardized inputs while preserving local temporal behavior. Overlapping windows are particularly useful because they increase the number of training samples and retain transitional response patterns that are important for classification (Literature Review, Section 3.1, Windowing & Segmentation) [7].

Feature engineering is another central theme in prior research. Instead of relying on raw sequences, many studies extract statistical, temporal, shape-based, and dynamic features from each signal window. Common descriptors include mean, variance, skewness, kurtosis, slopes, peak values, response time, and recovery time. Transformation-based methods such as FFT and DWT are also frequently used. This body of work consistently shows that engineered representations are especially effective when data are limited, because they reduce noise sensitivity while preserving the most discriminative aspects of the gas response (Literature Review, Section 3.2, Feature Extraction) [7].

The reviewed studies also indicate that no single machine learning model is universally best for gas classification. Classical models such as Logistic Regression, Support Vector Machines, Random

Forest, and Gradient Boosting are commonly compared because they offer a good balance between interpretability and predictive power. Logistic Regression is often used as a baseline, while SVM and ensemble methods are valued for their ability to capture more complex relationships. At the same time, prior work makes clear that greater model complexity does not automatically improve performance. In many gas sensing problems, the quality of the representation and the evaluation scheme matter more than the sophistication of the classifier itself (Literature Review, Section 4, Machine Learning Models for Gas Sensor Classification) [7].

Evaluation strategy is equally important. Because samples from the same experimental run are highly correlated, random splitting can lead to data leakage and overly optimistic results. To avoid this, researchers recommend run-level splitting and group-aware validation methods such as StratifiedGroupKFold. This ensures that performance is measured on genuinely unseen runs rather than on nearly identical observations from the same experiment. Another consistent point in prior work is that accuracy alone is not enough, especially when classes are imbalanced or difficult to separate. Precision, recall, and F1-score are therefore reported alongside accuracy to provide a more complete assessment of model behavior (Literature Review, Section 5, Evaluation Strategy for Gas Sensor Classification) [7].

Finally, gas sensing is presented in previous research as a pipeline problem rather than a single-model problem. A reliable workflow usually includes preprocessing, segmentation, feature extraction, data partitioning, model training, and evaluation, all arranged in a reproducible and traceable process. This perspective is important because the overall quality of the system depends on the interaction between these stages, not just on the classifier. Prior studies also highlight that multi-stage systems must be designed carefully to avoid error propagation, which makes modular pipeline design and rigorous validation essential in gas sensing applications (Literature Review, Section 6, Pipeline Design and Challenges in Gas Sensor Time-Series Analysis) [7].

The overall conclusion of this review is that effective gas sensing systems require careful preprocessing, temporal segmentation, feature engineering, leakage-safe validation, and appropriately chosen models. These findings form the conceptual basis for the methodology adopted in this report.

4. Business and Problem Understanding

The detection and identification of gases in livestock environments is an important challenge in modern agricultural and environmental monitoring systems. Volatile organic compounds (VOCs) released in such environments can provide valuable insights into animal health, hygiene conditions, and potential disease indicators. Early and accurate detection of these compounds can therefore contribute to improved animal welfare and more efficient farm management.

Traditional gas sensing approaches often rely on single-sensor measurements combined with threshold-based detection. While such methods are simple to implement, they cannot distinguish between gases with similar chemical properties and are highly sensitive to environmental noise. In real-world conditions, multiple gases are often present simultaneously, and sensor responses are influenced by factors such as temperature, humidity, and sensor drift. This leads to challenges such as cross-sensitivity, variability across measurements, and limited robustness.

To address these limitations, modern gas sensing systems adopt data-driven approaches in which sensor signals are treated as high-dimensional time series. Instead of relying on individual measurements, these systems analyze patterns in the sensor responses, often referred to as gas fingerprints. However, this shift introduces additional complexity, as the data becomes non-stationary and highly dependent on temporal dynamics.

Within this context, the objective of this project is to design and implement a robust and reproducible data and machine learning pipeline for gas sensing. The system employs a two-stage hierarchical modeling approach: first classifying single-gas versus mixture scenarios, then estimating probabilities for specific single gases (e.g., Toluene versus 2-butanone). It is expected to process raw sensor signals, extract meaningful representations through feature engineering on time-series windows, and produce reliable predictions about the presence and type of gases. In addition to predictive performance, the system must address practical requirements such as scalability, modularity, and reproducibility, ensuring that it can be used and extended in real-world applications.

A key challenge identified in this project is the high variability across experimental runs, which significantly impacts model generalization. Even under similar conditions, sensor responses can differ, making it difficult to learn stable patterns. This variability necessitates run-aware evaluation techniques, such as StratifiedGroupKFold, to prevent data leakage across runs. Furthermore, the temporal nature of the data implies that relevant information is not contained in individual measurements, but in the evolution of the signal over time. These challenges require careful design of both data processing and modeling strategies.

To tackle this problem, the project follows a structured approach inspired by the CRISP-DM methodology [1]. The problem is first analyzed in terms of domain characteristics and data properties, followed by the development of a feature-based modeling strategy tailored to temporal signal behavior. The final system integrates machine learning with MLOps components to ensure a complete and operational pipeline.

In summary, this project aims to bridge the gap between raw sensor data and actionable insights by developing a system that is not only accurate but also robust, scalable, and aligned with real-world constraints in gas sensing applications.

4.1 System Requirements and Design Goals

The design of the proposed system is driven not only by the gas sensing problem itself, but also by a set of practical system requirements that emerged from the characteristics of the dataset and the intended use of the solution. Since the project deals with non-stationery and variable-length sensor time-series collected across multiple experimental runs, the system must support both effective machine learning and reliable operational behavior.

Several design goals guided the development of the pipeline. First, the system should support end-to-end automation, covering the full workflow from raw data ingestion to prediction generation. Second, reproducibility is a central requirement, meaning that data transformations, feature generation, model training, and evaluation should be repeatable under the same conditions. Third, the system should be modular and maintainable, allowing individual pipeline stages to be updated or reused without affecting the entire workflow.

In addition, the design must support scalability, so that the pipeline can handle additional datasets or experimental runs without requiring major structural redesign, leveraging DVC for data versioning and batch processing. Robustness to data variability is also essential, as sensor responses differ significantly across runs and experimental conditions. Furthermore, the system should promote generalization across runs by preventing leakage between correlated samples during model development and evaluation, using techniques like StratifiedGroupKFold. Finally, deployment readiness is considered an important design objective, so that trained models can be exposed through a programmatic API interface and the overall system can be executed consistently across environments.

These requirements directly influence the architectural and modeling choices made in the remainder of this project, including the use of an artifact-driven pipeline with DVC, group-based validation, modular pipeline stages, and MLOps components such as Airflow for orchestration, experiment tracking, versioning, and deployment.

4.2 Functional and Non-Functional Requirements

4.2.1 Functional Requirements

- The system is expected to provide the following core functionalities:
- Ingest raw time-series sensor data from multiple experiments
- Preprocess and normalize sensor signals at the run level
- Transform variable-length time series into fixed-size windows
- Extract statistical, temporal, and shape-based features (e.g., mean, variance, and temporal derivatives as detailed in notebook 02) [9]
- Train machine learning models for gas classification (e.g., Logistic Regression, SVM, Random Forest, and Gradient Boosting from notebooks 03 and 04)
- Support a two-stage modeling approach (mixture detection and gas classification/probability estimation)
- Evaluate models using group-based validation (e.g., StratifiedGroupKFold to prevent run leakage)
- Store intermediate and final outputs as reusable artifacts (using DVC)
- Track experiments, including parameters, metrics, and models
- Expose trained models through an API for prediction (via the api/ module)

4.2.2 Non-Functional Requirements

In addition to functionality, the system must satisfy several quality requirements:

- Reproducibility: All experiments must be repeatable with the same data and configuration (enabled by DVC and GitLab CI)
- Scalability: The system should handle increasing data volumes and additional experiments (via batch processing and DVC)
- Modularity: Pipeline components should be independent and reusable
- Maintainability: The system should be easy to extend and modify
- Robustness: The system should handle noisy and variable data without failure
- Traceability: All data transformations and model results should be trackable (via DVC and logs)
- Performance: The pipeline should execute efficiently within reasonable time constraints (orchestrated by Airflow)
- Deployment-readiness: The system should support execution in different environments using containerization (e.g., Docker and AWS ECS)

4.3 User Stories

To better capture stakeholder needs and system usage, the requirements are translated into user stories representing different roles.

Table 1: User Stories and Acceptance Criteria for the MLOps Pipeline Stages

ID	Title	Definition	Acceptance Criteria
US-01	Data Ingestion and Preprocessing	As a Data Engineer, I want raw sensor files to be ingested and preprocessed automatically so that the pipeline is repeatable and does not rely on manual steps	1. Running the pipeline ingests raw data and produces preprocessed outputs successfully. 2. Missing or invalid input data produces a clear error and fails fast. 3. Preprocessed outputs are saved as reusable artifacts for downstream stages.
US-02	Feature Engineering for Modeling	As a Data Scientist, I want structured feature datasets to be generated from time-series windows so that I can train and compare models consistently	1. Feature datasets include required metadata (e.g., run-level identifiers) and engineered numeric features. 2. Stage-1 feature dataset and Stage-2 single-gas subset are both generated and saved. 3. Feature outputs include basic quality checks (shape, schema, and missing-value summary).
US-03	Stage-1 Model Training with Leakage-Safe Validation	As an ML Engineer, I want Stage-1 models to be trained and compared using run-aware validation so that model selection is robust and realistic	1. Cross-validation is group-aware (run-level split, leakage-safe). 2. Accuracy, precision, recall, and F1 are reported for all baseline models. 3. The selected Stage-1 model and its metadata are saved as deployable artifacts.

ID	Title	Definition	Acceptance Criteria
US-04	Stage-2 Probability Modeling	As an ML Engineer, I want Stage-2 to be trained on true single-gas data and produce class probabilities so that downstream decisions can be confidence-aware	1. Stage-2 training uses only the single-gas subset. 2. Baseline and tuned model results are recorded and comparable. 3. Final Stage-2 output includes calibrated class probabilities for target gases.
US-05	Artifact Storage with MinIO/S3-Compatible Backend	As an MLOps Engineer, I want datasets, models, and reports stored in object storage so that artifacts are centralized, versionable, and portable across environments	1. MinIO is configured and accessible for local/CI artifact storage. 2. Pipeline artifacts (processed data, model bundles, reports, predictions) are persisted to object storage. 3. Artifact paths and versions are traceable and reproducible across runs.
US-06	CI/CD and Quality Gates	As a Team Lead, I want automated CI/CD pipelines so that code quality, reproducibility, and delivery are enforced continuously	1. CI runs tests and required validation checks automatically on commits/merge requests. 2. Build/deploy workflow is automated and produces consistent outputs. 3. Failed checks block promotion until issues are resolved.
US-07	Hosted, Containerized Deployment	As a System User, I want the platform and API to run in a hosted environment so that predictions and pipeline services are accessible beyond local development	1. Services run in containers with consistent environment configuration. 2. Core components (orchestration, tracking/storage, API) are reachable in the hosted setup. 3. Deployment process is documented and repeatable.
US-08	Notebook and Reporting Deliverables	As a Project Stakeholder, I want clear notebooks and a final	1. Notebooks for feature engineering, Stage-1, and Stage-2 are complete and aligned with implemented pipeline behavior.

ID	Title	Definition	Acceptance Criteria
		technical report so that methodology, results, and decisions are transparent and auditable	2. Reported metrics and model-selection statements match current notebook/pipeline results. 3. Final report includes requirements traceability, architecture rationale, evaluation method, and deployment summary.

Table 2: Definition of Done (DoD) and Completion Criteria for Project

ID	Project phase	Parameters
01	Stage 1 (Single-Gas vs Mixture Classification)	1. Input dataset is successfully loaded from the engineered feature artifact and passes basic schema checks. 2. Target label (target_mixture) and run-level grouping (run_id) are correctly created and used. 3. Model evaluation is performed with leakage-safe, run-aware validation (StratifiedGroupKFold). 4. Baseline model comparison is completed with reported metrics: Accuracy, Precision, Recall, and F1-score. 5. Final Stage-1 model is selected based on the defined selection rule and saved as a deployable model bundle. 6. Stage-1 outputs are persisted as artifacts: a. model bundle (model + metadata), b. model comparison report, c. CV metrics report, d. training predictions artifact. 2. 7. Experiment/logging artifacts are traceable and reproducible (same config and input produce consistent outputs).
02	Stage 2 (Single-Gas Probability Modeling)	1. True single-gas subset is generated from Stage-1 preparation logic and saved as a dedicated artifact. 2. Stage-2 target definition and feature matrix are validated for schema consistency and completeness. 3. Run-aware validation is applied for all Stage-2 experiments to prevent leakage across runs. 4. Baseline and tuned candidates are evaluated and compared using the same metric framework. 5. Final Stage-2 model is selected and produces class probabilities for target gases (Toluene vs 2-butanone). 6. Stage-2 outputs are persisted as artifacts: a. final model bundle (model + metadata), b. baseline/tuning comparison reports, c. CV metrics report, d. training predictions artifact.

ID	Project phase	Parameters
		7. Final selected model and results are aligned with notebook analysis and pipeline configuration.
03	MLOps / Delivery	1. End-to-end pipeline can be executed reproducibly in the project environment. 2. Artifacts are versioned/stored in the configured storage flow (local/MinIO/S3-compatible pathing). 3. CI/CD checks pass for the committed version (tests/validation gates configured by the project). 4. API serving layer is operational for prediction endpoints. 5. Final report, notebooks, and pipeline outputs are mutually consistent (no contradiction in model-selection claims or key metrics).

4.4 Requirements Traceability

To ensure alignment between system requirements and implementation, a traceability mapping is established between each requirement and the enabling tools and engineering techniques.

1. Reproducibility

Reproducibility is achieved through DVC for dataset and pipeline versioning, MLflow for experiment/run tracking and model artifact lineage, and containerization with Docker to keep runtime dependencies and execution behavior consistent across development, CI, and deployment.

2. Scalability

Scalability is supported by a modular stage-based pipeline architecture and workflow orchestration with Airflow, enabling the system to process increasing data volumes and incorporate additional workflow stages with minimal redesign.

3. Modularity and Maintainability

Modularity and maintainability are ensured through separation of concerns and independent pipeline stages (ingestion, feature engineering, training, evaluation, inference), so individual components can be updated or replaced without destabilizing the full system.

4. Traceability

Traceability is implemented via an artifact-driven workflow, where intermediate and final outputs are persistently stored and versioned, enabling auditability, stage-level inspection, and reproducible reruns.

5. Deployment Readiness

Deployment readiness is enabled through a FastAPI serving layer and Docker-based packaging/deployment, providing consistent behavior across local, testing, and production environments.

Overall, this mapping demonstrates that the pipeline's architectural and implementation choices directly satisfy the core system requirements.

5. Process Management

The project was developed using an Agile and iterative approach, structured around incremental development cycles as outlined in the team charter (docs/team_charter.md).

Work was organized into short iterations, documented in sprint iterations (docs/sprint_iterations/), where each iteration focused on delivering a specific part of the pipeline, such as data preprocessing, feature engineering, modeling, or MLOps integration. This allowed continuous validation of results and early identification of issues.

Tasks were defined and tracked throughout the project to ensure clear ownership and progress monitoring. The development process emphasized modular implementation, enabling different components of the pipeline to be developed and tested independently.

Regular feedback loops were incorporated during development. Intermediate results, including data analysis findings, model performance, and pipeline outputs, were continuously evaluated and refined. This iterative feedback process helped align technical decisions with project goals and data characteristics.

Version control was used to manage code changes systematically with Git, and DVC for data versioning, ensuring traceability and maintainability. Changes were integrated incrementally to reduce integration risks and maintain system stability.

Overall, the process focused on incremental delivery, continuous validation, and adaptability, which are essential in data-driven projects where insights evolve throughout the development lifecycle.

6. Data Understanding

A comprehensive understanding of the dataset is essential for designing an effective machine learning pipeline, particularly in gas sensing applications, where data characteristics strongly influence model performance. The dataset used in this project consists of time-series sensor measurements collected from multiple experiments, each representing different gas exposure scenarios, as explored in notebook 01 [8].

Each experimental run contains a sequence of sensor readings over time from multiple sensors, capturing the dynamic response (gas fingerprints) of the sensor to gas exposure. Unlike static datasets, these signals exhibit a clear temporal structure, where meaningful information is embedded in the evolution of the signal rather than in individual observations. As a result, the dataset must be treated as inherently time-dependent, requiring approaches capable of capturing temporal patterns.

Several key characteristics of the dataset were identified during the exploratory analysis phase in notebook 01 [8]. First, the data show significant variability across experimental runs. Even under similar conditions, sensor responses can differ considerably, indicating the presence of run-to-run variability. This variability presents a major challenge for model generalization, as patterns learned from one run may not directly transfer to another.

Second, the dataset exhibits non-stationary behavior, reflected in changing signal characteristics over time. The sensor response typically follows distinct temporal phases, including a baseline period, an exposure phase (comprising both rise and plateau behavior), and a recovery phase. Each of these phases is characterized by different statistical properties, reinforcing the importance of modeling temporal dynamics rather than relying on static representations.

Third, the sensor signals exhibit relatively low levels of noise, but still contain minor measurement variability. While light smoothing techniques can be applied to reduce small fluctuations, it is critical to preserve the underlying temporal structure of the signal to avoid losing important information. This creates a trade-off between noise reduction and information preservation.

Another important characteristic is the variation in signal scale across different experiments. Some signals exhibit significantly larger magnitudes than others, which can bias machine learning models if not properly normalized. Additionally, the length of time-series sequences varies across runs, making it difficult to directly apply models that require fixed-size inputs. To address this, the data is transformed into fixed-size segments using a windowing approach.

Class imbalance is also present in the dataset, with some experimental conditions being more represented than others. This imbalance can influence both model training and evaluation, particularly when using standard performance metrics.

An important observation from the analysis is that temporal patterns are more informative than spatial characteristics. The discriminative information is primarily encoded in how the signal evolves over time, rather than in isolated values. This insight directly motivates the use of temporal feature extraction techniques in subsequent stages of the pipeline.

These observations lead to several important implications for the system design. First, models must be able to capture temporal dependencies, as relevant information is distributed over time. Second, preprocessing techniques are required to normalize signal scales while preserving signal structure. Third, variable-length sequences must be transformed into a consistent representation suitable for machine learning models. Finally, evaluation must explicitly account for run-level dependencies, as samples originating from the same run are highly correlated and can lead to data leakage if not properly handled.

Overall, the data understanding phase highlights that the primary challenge of this project lies not only in classification, but in the reliable extraction of meaningful patterns from complex, variable, and temporally structured sensor data.

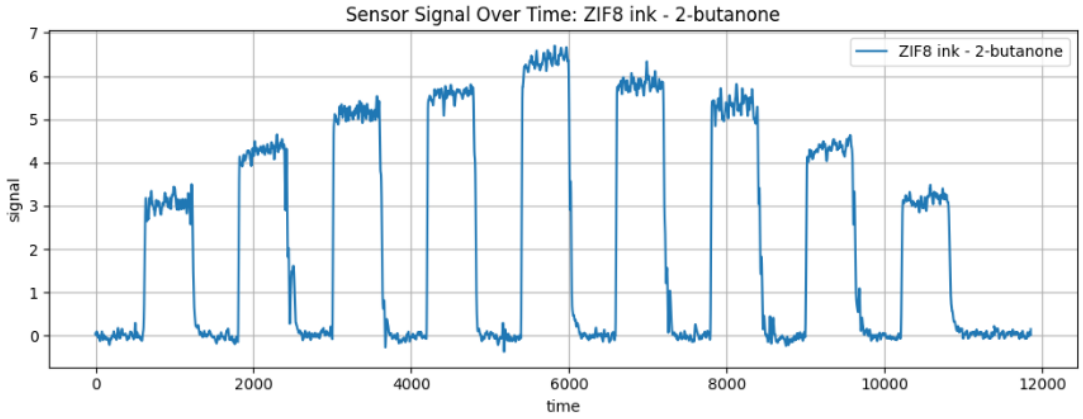


Figure 1:Representative time-series sensor signal illustrating the temporal evolution of the sensor response over time

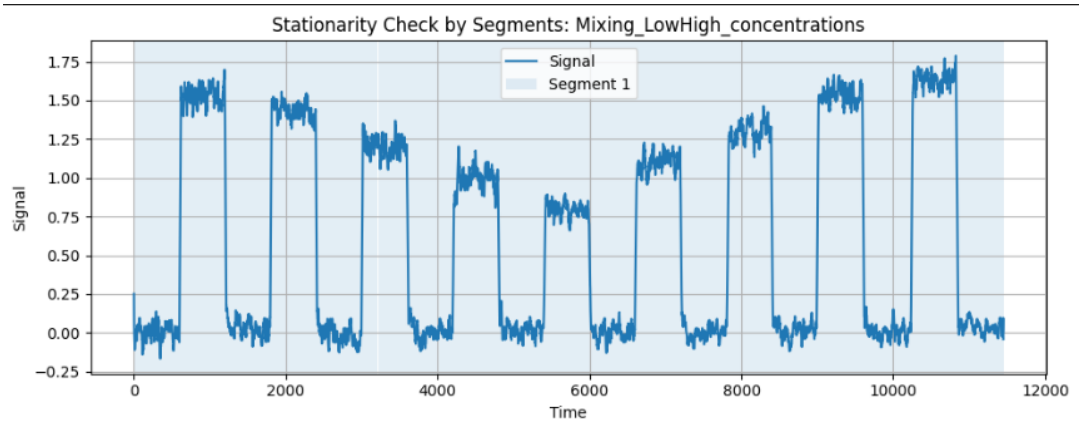


Figure 2: Segment-based analysis of sensor signals illustrating non-stationary behavior and changes in statistical properties over time

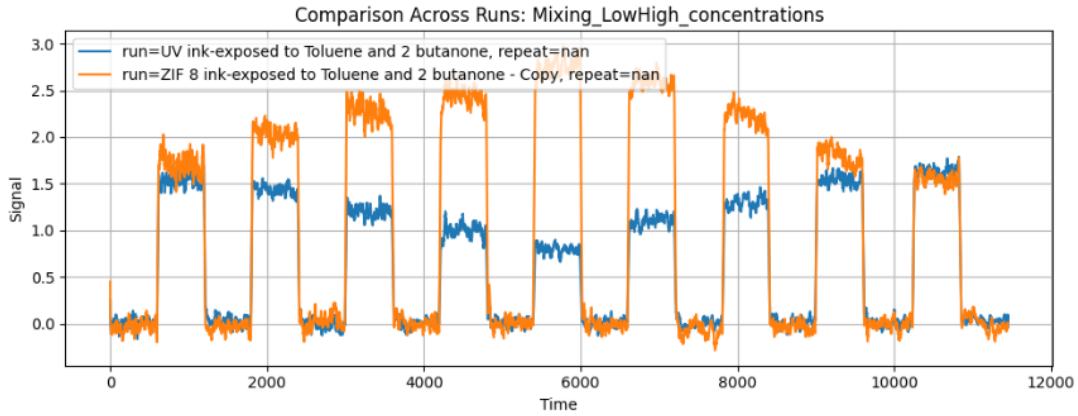


Figure 3: Comparison of sensor signals across different experimental runs, illustrating significant variability in signal patterns

7. Data Preparation and Feature Engineering

Based on the insights obtained during the data understanding phase, a structured data preparation and feature engineering pipeline was designed to transform raw sensor signals into representations suitable for machine learning models. Given the characteristics of the dataset, particularly temporal dependencies, variability across runs, and differences in signal scale, multiple preprocessing steps are required to ensure consistency, robustness, and meaningful feature extraction.

The first step in the pipeline is normalization of the sensor signals. Due to significant variation in signal magnitude across different experimental runs, normalization is applied at the run level using a Min-Max scaling approach. This ensures that all signals are transformed to a comparable range, preventing models from being biased toward experiments with larger absolute values and enabling the learning process to focus on relative patterns rather than magnitude differences.

Although the sensor signals are relatively clean, a light smoothing technique is optionally applied using a centered rolling mean with a small window size. The purpose of this step is to reduce minor fluctuations while preserving the overall temporal structure and dynamics of the signal. The use of a small window ensures that important transient behaviors are not removed through oversmoothing.

A key challenge in the dataset is the variability in sequence length across different runs. Since most machine learning models require fixed-size inputs, a window-based transformation is applied. Each time-series signal is segmented into overlapping fixed-size windows (for example, a window size of 100 with a step size of 50). This approach allows the model to capture local temporal patterns while ensuring a consistent input representation. The use of overlapping windows also increases the number of training samples and enables better representation of transitional signal behavior.

Instead of using raw signals directly, a feature-based representation is adopted to provide a compact and structured input suitable for classical machine learning models, particularly given the relatively limited size of the dataset. This approach transforms each window into a set of descriptive features that capture different aspects of the signal, as implemented in notebook 02 [9].

The extracted features are designed to reflect statistical, temporal, and structural properties of the signal. Statistical features such as mean, standard deviation, minimum, maximum, median, and range summarize the distribution of values within each window. Temporal features, including trend slope and first-order differences, capture the direction and rate of change of the

signal. Shape-based features, such as peak value, peak position, rise strength, and fall strength, characterize the structural behavior of the signal.

In addition to these, higher-order and dynamic features are included to provide a richer representation. These include signal energy, skewness, kurtosis, flatness, zero-crossing rate of the signal differences, as well as positional features such as signal start and end values and relative peak position. Together, these features capture both global and local properties of the signal.

This feature design is directly motivated by the observation that temporal patterns are more informative than absolute signal magnitudes. By focusing on how the signal evolves over time rather than relying on raw values alone, the feature representation improves the separability of different gas responses in the feature space. This representation is also consistent with the concept of gas fingerprints, where distinct gases produce characteristic response patterns.

Finally, to ensure a realistic and reliable evaluation setup, the dataset is split based on experimental runs rather than individual samples. This run-aware splitting strategy prevents data leakage, as samples originating from the same run are highly correlated. By separating entire runs into training and testing sets and using group-aware validation such as StratifiedGroupKFold, the evaluation setup better reflects real-world scenarios, where models must generalize to unseen experimental conditions.

Overall, the data preparation and feature engineering pipeline transforms complex, variable-length, and temporally structured sensor signals into consistent and informative representations, enabling the effective application of machine learning models.

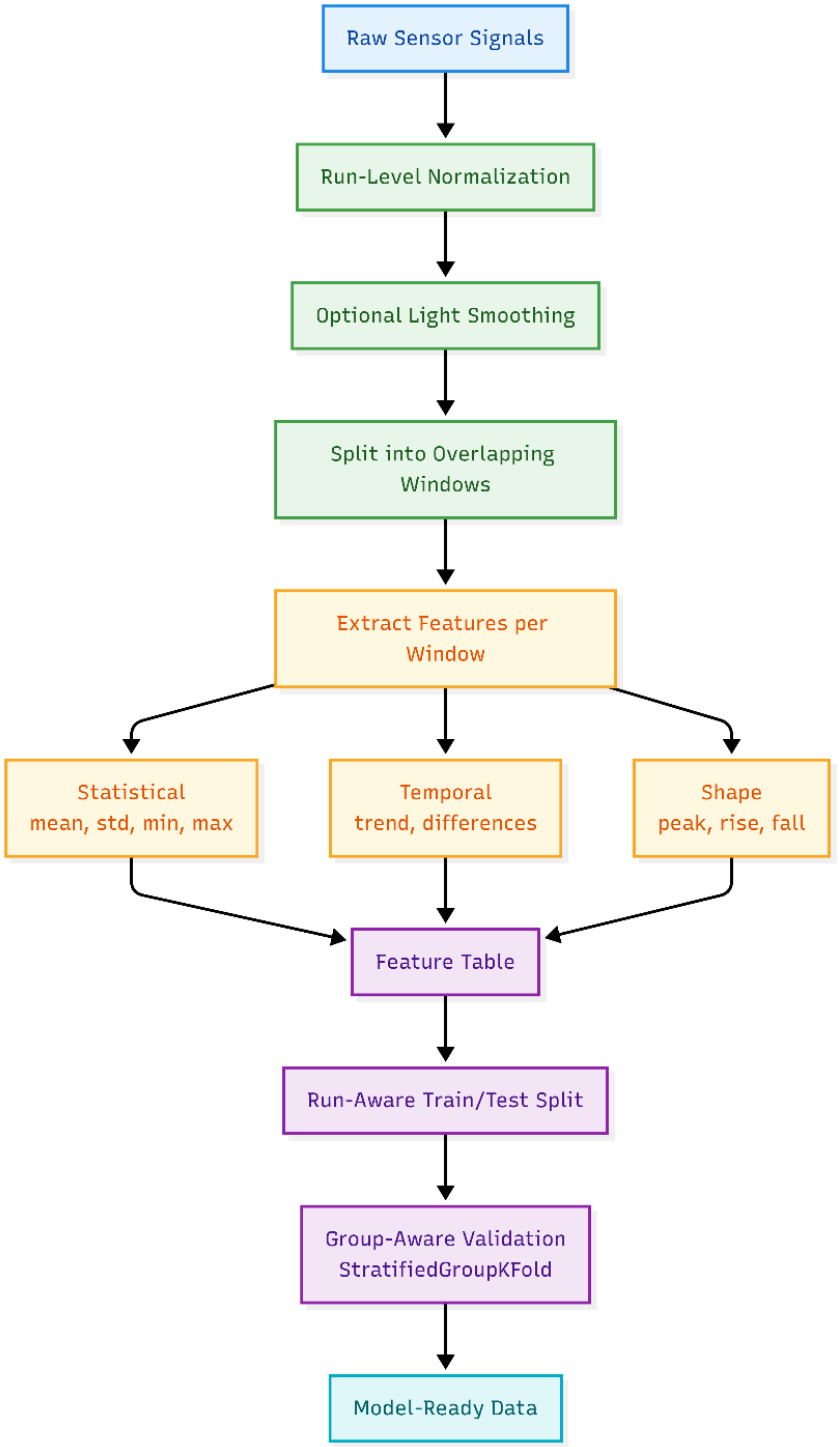


Figure 4: End-to-End Data Preprocessing and Feature Engineering Pipeline

8. Modeling

The modeling phase aims to learn meaningful patterns from the engineered feature set to accurately classify gas-related conditions. Given the complexity of the dataset, particularly the high variability across experimental runs, overlapping signal characteristics between classes, and limited data size, a structured and robust modeling strategy is required.

To address these challenges, a two-stage modeling approach is adopted. Instead of directly performing full multi-class classification, the problem is decomposed into two sequential tasks. In the first stage, a binary classification model is trained to distinguish between mixture and non-mixture conditions. This step simplifies the problem by isolating complex mixture scenarios, which exhibit more variability and less separability.

In the second stage, a second model is trained on a filtered subset of the dataset containing only non-mixture conditions. This stage focuses on discriminating individual single gases, with the final output interpreted as class probabilities for the remaining gas types (in the implementation, the emphasis is on distinguishing Toluene versus 2-butanone). By focusing on this subset, the model can learn more specific and discriminative patterns associated with individual gases without being affected by the additional complexity introduced by mixtures.

This decomposition is motivated by the observation that mixture detection and gas identification involve different levels of complexity and benefit from specialized modeling strategies. Separating these tasks allows the system to capture hierarchical relationships in the data while improving interpretability and robustness.

A feature-based modeling approach is adopted instead of using raw time-series signals directly. This decision is primarily driven by the relatively limited size of the dataset and the presence of variability across experimental runs. Feature extraction provides a compact and structured representation of the data, capturing key statistical, temporal, and structural characteristics while reducing sensitivity to noise and minor fluctuations. This enables classical machine learning models to perform effectively without requiring large amounts of training data.

Several machine learning models are evaluated to determine the most suitable approach for both stages. These include Logistic Regression, Support Vector Machines (SVM), Random Forest, and Gradient Boosting. Logistic Regression serves as a baseline model and is effective when classes are approximately linearly separable. SVM is capable of handling more complex decision boundaries, particularly in high-dimensional feature spaces. Tree-based models, including Random Forest and Gradient Boosting, are well-suited for capturing non-linear relationships and interactions between features.

Model evaluation is performed using cross-validation with run-level grouping, as implemented in notebooks 03 and 04. This ensures that samples originating from the same experimental run do not appear in both training and validation sets, preventing data leakage and providing a more realistic estimate of model performance. This evaluation strategy is particularly important given the strong correlation between samples within the same run.

Experimental results under run-aware cross-validation indicate that Random Forest provides the strongest overall Stage-1 performance, achieving 96.1% accuracy and F1-score of 0.96. Tree-based models (Random Forest and Gradient Boosting) significantly outperform the Logistic Regression baseline under group-aware validation (Notebook 03, Section 4.5) [10]. Tree-based models (Random Forest and Gradient Boosting) remain competitive and show strong class-specific behavior, but they do not consistently outperform the Logistic Regression baseline on the current engineered feature space. SVM offers additional non-linear flexibility, yet its overall performance remains below the top-performing baseline in this setting.

Dimensionality reduction using Principal Component Analysis (PCA) is also explored to assess whether reducing feature space complexity improves model performance. However, the results indicate that PCA does not lead to significant improvements, suggesting that the engineered features already capture the most relevant information in a compact form.

Neural network models are briefly investigated as an alternative approach. However, convergence issues and unstable performance are observed during training, likely due to the limited size of the dataset and high variability across experimental runs. As a result, classical machine learning approaches are retained as the primary modeling strategy.

A key challenge throughout the modeling phase is achieving generalization across experimental runs. Due to run-to-run variability, models that perform well on certain runs may not generalize effectively to unseen conditions. This reinforces the importance of both feature design and evaluation strategy in building a robust system.

Overall, the modeling approach is designed to balance complexity, interpretability, and robustness. By combining a two-stage architecture with carefully selected models and a feature-based representation, the system effectively addresses the challenges posed by the dataset while maintaining generalizability to new experimental scenarios.

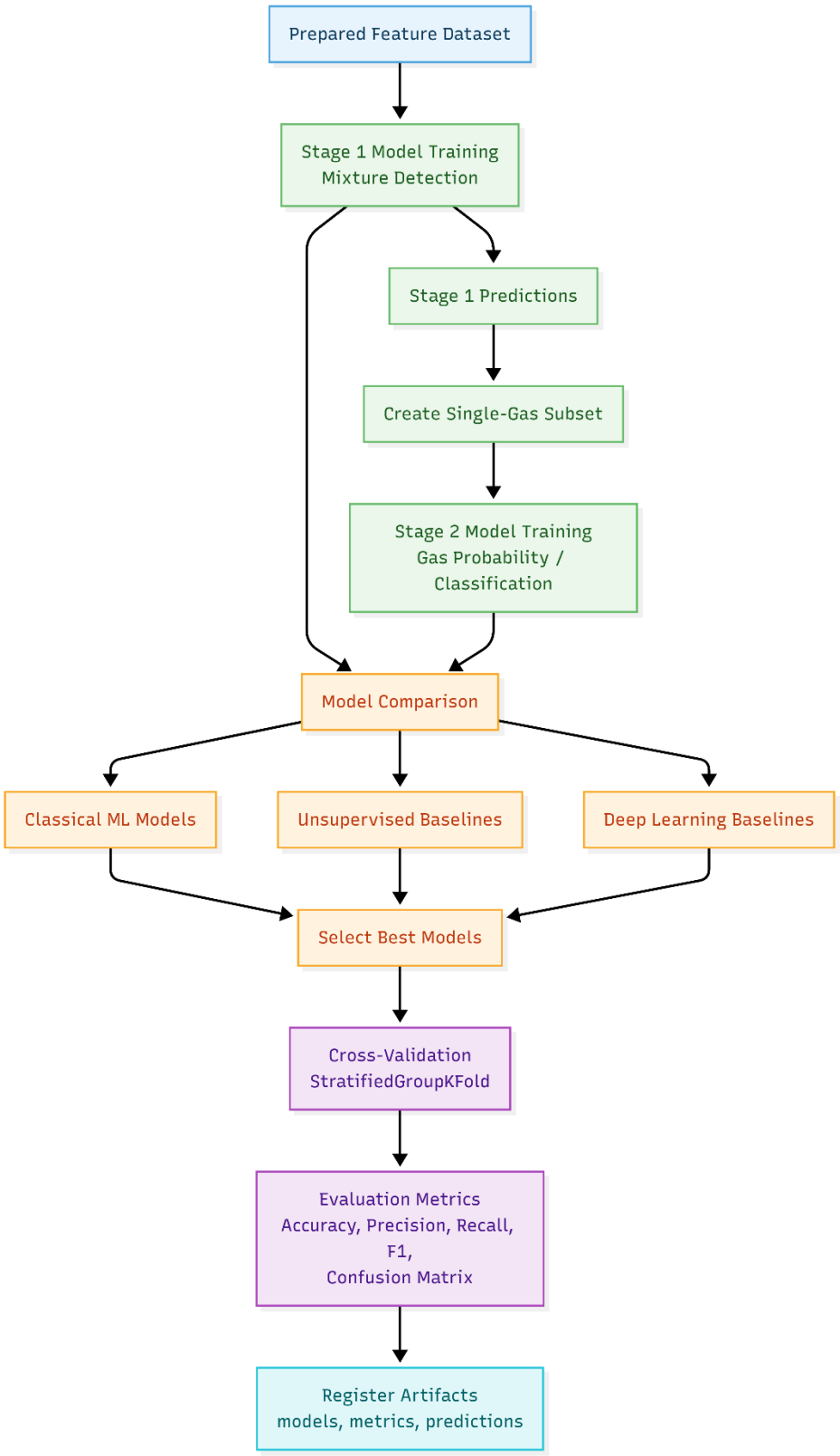


Figure 5: End-to-End MLOps Workflow for Hierarchical Model Training, Validation, and Artifact Registration

9. Evaluation

The evaluation phase focuses on assessing the performance of the proposed models realistically and reliably. Given the characteristics of the dataset, particularly the strong correlation between samples originating from the same experimental run, careful consideration is required to avoid overly optimistic performance estimates.

A key component of the evaluation strategy is the use of group-based cross-validation. Instead of randomly splitting individual samples, the dataset is partitioned based on experimental runs, ensuring that all samples from a given run are assigned exclusively to either the training or validation set. This approach prevents data leakage, as samples within the same run share similar temporal patterns and statistical properties. In contrast, random sample-level splitting would likely result in significantly higher but misleading performance estimates due to implicit information leakage.

The evaluation is conducted separately for both stages of the modeling pipeline. In the first stage, performance is assessed for the binary mixture detection task, while in the second stage, evaluation focuses on gas classification using the filtered subset of non-mixture data, interpreted as class probabilities for the remaining single gases (with emphasis on Toluene versus 2-butanone in the implemented work). This separation allows a clearer understanding of model behavior across different levels of task complexity.

Using this evaluation setup, multiple models were assessed for both stages of the pipeline. In Stage 1, performance is strong under run-aware cross-validation, with Random Forest achieving the best overall mean accuracy (96.1%) and mean F1-score (0.96) among tested models. In Stage 2, performance is lower than Stage 1 due to feature-level overlap between Toluene and 2-butanone constraining discrimination; however, tuned Logistic Regression (C=0.01, no class_weight) provides the best final Stage-2 results with 79% accuracy and F1-score of 0.62 (Notebook 04, Section 6) [11]. These outcomes show that robust, leakage-aware evaluation is essential for reliable model selection.

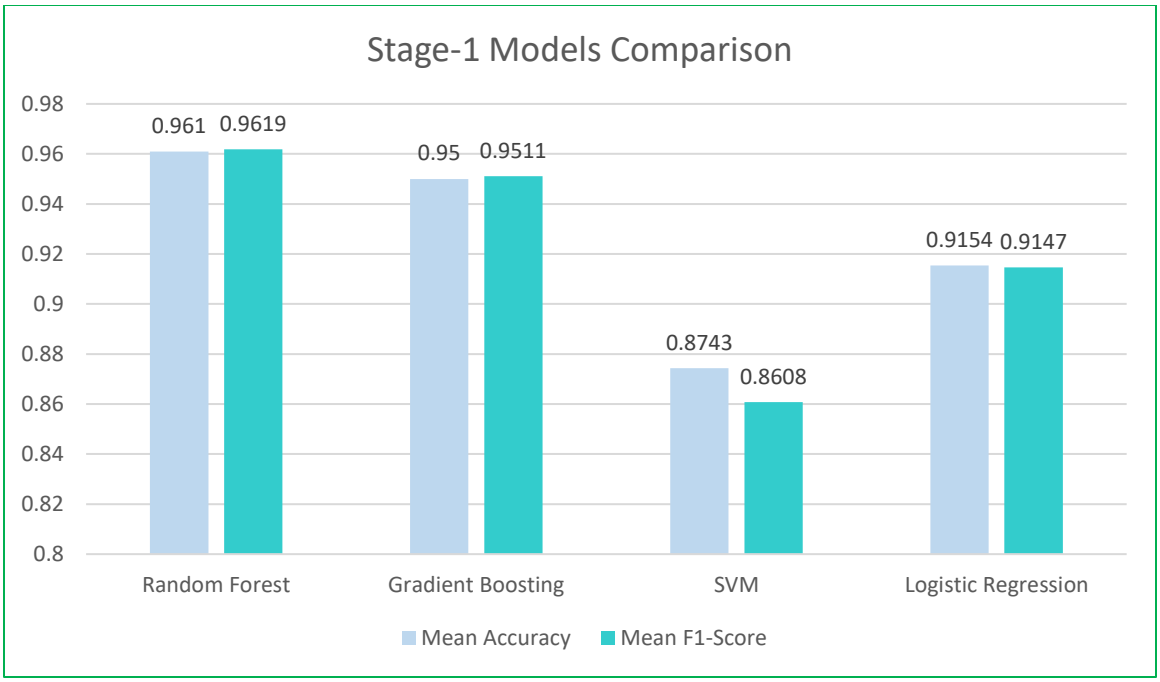


Figure 6:Stage-1 Models Comparison

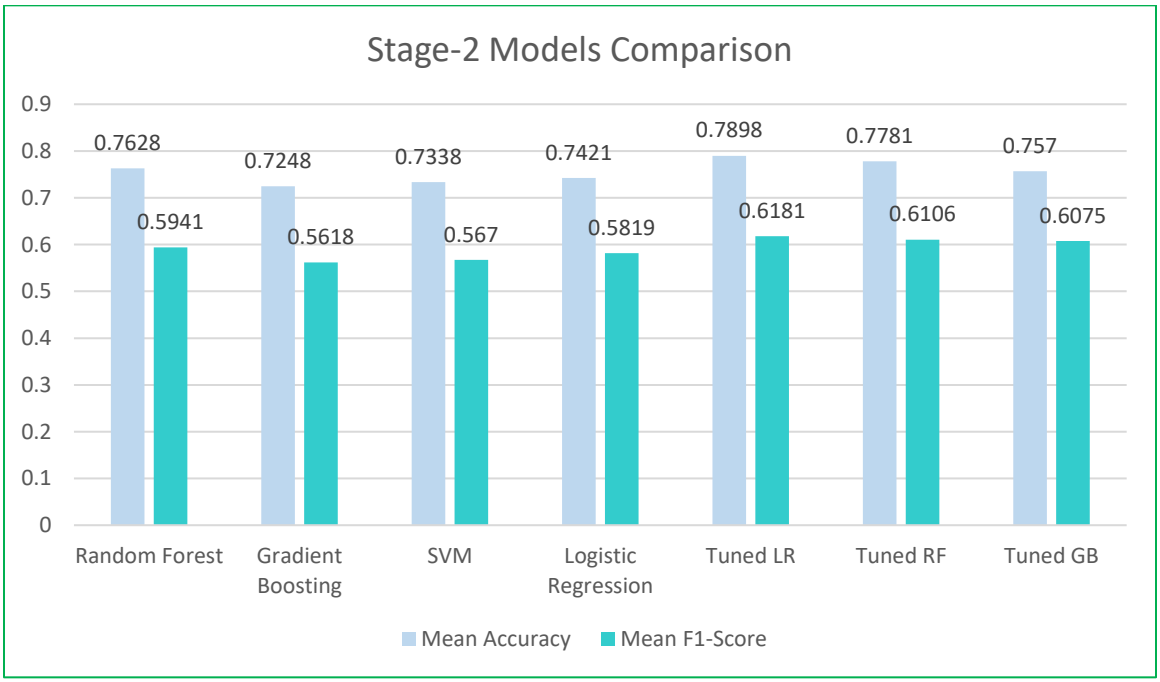


Figure 7:Stage-2 Models Comparison

In addition to relatively low average performance, the results exhibit considerable variability across validation folds. This high variance indicates instability in model performance, which is primarily caused by differences between experimental runs. Models that perform well on certain runs may not generalize effectively to others, highlighting the difficulty of achieving consistent performance across varying conditions.

These findings suggest that the observed performance limitations are not solely due to model choice but are largely driven by inherent characteristics of the dataset. High run-to-run variability, non-stationary signal behavior, and overlapping patterns between classes contribute to the difficulty of the classification task. In particular, distribution shifts between experimental runs significantly reduce model generalization capability.

Additional experiments are conducted to evaluate the impact of dimensionality reduction and alternative modeling strategies. Dimensionality reduction using Principal Component Analysis (PCA) does not lead to performance improvements and, in some cases, results in slight performance degradation. This suggests that the engineered feature set already captures the most relevant information and that further compression may remove useful signals.

Neural network models are also explored; however, they exhibit convergence issues and unstable performance during training. These limitations are likely due to the relatively small dataset size and the high variability across runs. As a result, classical machine learning models remain the most suitable choice for this problem setting.

Overall, the evaluation demonstrates that the proposed system provides a realistic and reliable assessment of model performance. By enforcing run-aware splitting, the reported metrics reflect true generalization behavior rather than optimistic leakage-driven estimates. Stage-1 results are strong and stable, while Stage-2 remains more challenging because class boundaries are subtler and run-to-run variability is higher. This distinction is expected and supports the use of a two-stage architecture for improved task specialization.

10. System Design and MLOps Architecture

In addition to developing accurate predictive models, this project focuses on designing a complete and reproducible system for processing, training, and deploying machine learning models. To achieve this, a modular and scalable architecture is developed, integrating multiple components that collectively support the full lifecycle of the data and machine learning pipeline.

The system follows an end-to-end pipeline structure, starting from data ingestion and progressing through preprocessing, feature engineering, model training, evaluation, and prediction. Each stage is designed as an independent and reusable component, enabling flexibility, maintainability, and ease of extension. This modular design allows individual components to be modified or improved without affecting the overall system.

A key design principle adopted in this project is the use of an artifact-driven pipeline. Instead of passing data directly between components in memory, intermediate results are stored as artifacts, such as processed datasets, feature matrices, trained models, and evaluation outputs. These artifacts are persisted using structured file formats and managed with DVC, allowing different stages of the pipeline to operate independently while ensuring reproducibility and traceability.

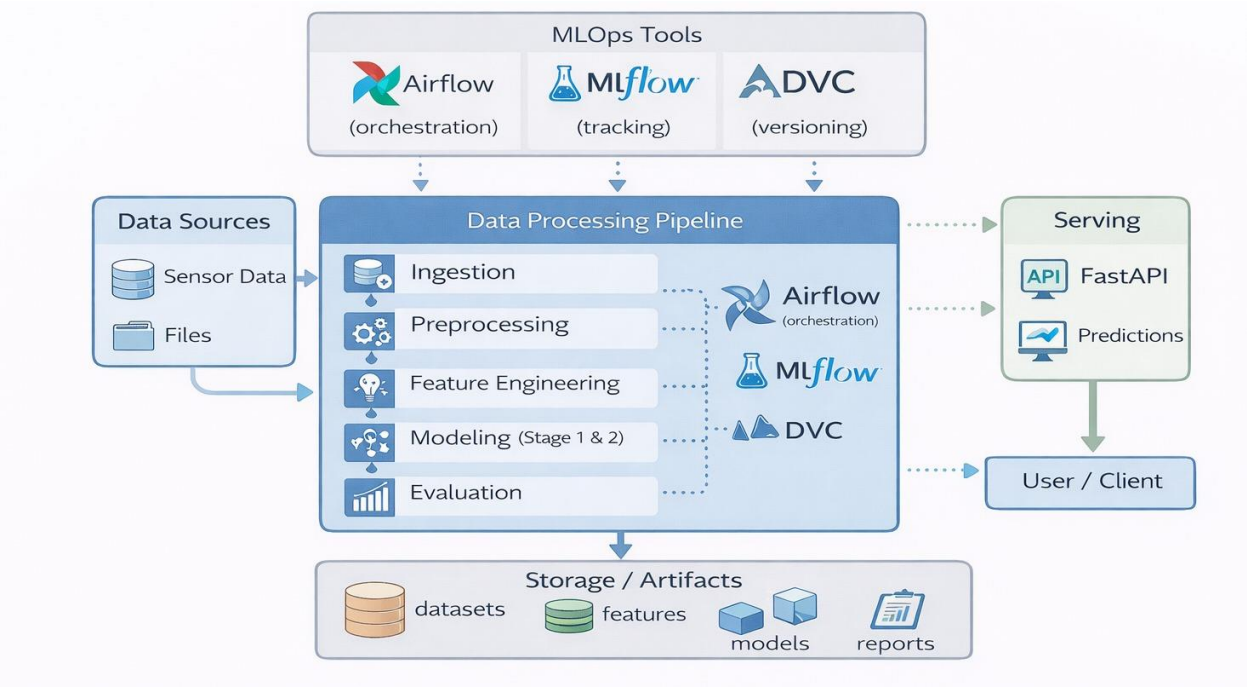


Figure 8: End-to-End MLOps Architecture for Sensor Data Processing, Modeling, and Serving

10.1 Architecture Components Overview

To better understand the structure of the system, the overall architecture can be decomposed into several core components. Each component is responsible for a specific aspect of the pipeline, contributing to modularity, scalability, and maintainability.

Table 3: Overview of Core Components in the MLOps Architecture

Component	Description	Technology
Data Processing	Ingestion, parsing, and preprocessing of raw time-series data	Python
Feature Engineering	Windowing of time-series data and extraction of statistical, temporal, and structural features	Python
Modeling	Two-stage modeling (mixture detection and single-gas probability estimation) with training and evaluation	Scikit-learn (Logistic Regression, SVM, Random Forest, Gradient Boosting)
Orchestration	Pipeline orchestration, scheduling, and task dependency management	Apache Airflow + GitLab CI
Experiment Tracking	Tracking experiments, metrics, parameters, and models	MLflow
Data Versioning	Version control for datasets, features, and intermediate artifacts	DVC
API Service	Serving trained models and predictions via REST API	FastAPI
Containerization	Containerized environment for reproducible and consistent execution	Docker
Artifact Storage	Storage of intermediate datasets, trained models, and experiment artifacts	S3-compatible object storage with MinIO locally and AWS S3 in production, integrated with DVC and MLflow

To orchestrate the execution of the pipeline, Apache Airflow is used as the workflow management system. Airflow enables the definition of the pipeline as a directed acyclic graph (DAG), where each task corresponds to a specific stage in the workflow. This approach provides clear visibility into task dependencies, supports automated scheduling, and allows for robust error handling and retry mechanisms.

The pipeline is composed of multiple stages, including data ingestion, preprocessing, feature engineering, model training, evaluation, and prediction. Each stage operates on artifacts generated by previous stages, ensuring loose coupling between components and enabling partial re-execution when needed. The Airflow DAG implementation in gas_sensor_ml_pipeline_dag.py orchestrates these stages, while validation and trigger scripts in scripts help ensure the workflow is executable and reproducible.

10.2 Artifact-Driven Pipeline Design

A central design decision in this system is the adoption of an artifact-driven pipeline. Instead of relying on in-memory data transfer between tasks, each stage produces and consumes well-defined artifacts stored on disk.

These artifacts include:

- Processed datasets
- Windowed time-series data
- Extracted feature sets
- Trained model files
- Evaluation reports and metrics
- Prediction outputs

In this project, artifact management is supported by DVC, which tracks dataset versions, feature tables, and model artifacts through dvc.yaml and dvc.lock, while the pipeline logic in the top-level pipelines module reads and writes these artifacts consistently. This approach improves reproducibility, as each stage can be rerun independently using the same input artifacts. It also enhances traceability, allowing intermediate results to be inspected and validated at each step of the pipeline.

10.3 Experiment Tracking and Data Versioning

Experiment tracking and model management are handled using MLflow. This component records model configurations, parameters, metrics, and artifacts, enabling systematic comparison of different experiments. By maintaining a centralized record of experiments, MLflow supports reproducibility and facilitates informed decision-making during model selection.

Data versioning is managed using DVC, which ensures that different versions of raw datasets, transformed features, intermediate results, and models are tracked over time. This is particularly important in machine learning workflows, where changes in data can significantly impact model

performance. By versioning data alongside code, the system maintains consistency between experiments and results.

10.4 Model Serving and Deployment

For model serving, a FastAPI-based service is implemented to expose trained machine learning models through a RESTful API. This service provides endpoints for health checks, model information, and prediction requests, enabling structured interaction with the system.

The API is containerized using Docker, ensuring consistent behavior across different environments. This allows the model serving component to be easily deployed alongside other system components such as Airflow, MLflow, and MinIO within a unified containerized architecture.

The repository also includes a GitLab CI job for AWS deployment, where the API image is built, pushed to ECR, and the ECS service is updated. This demonstrates that the system can be deployed in an AWS-hosted environment, not just a local development setup.

Overall serving approach provides a practical and extensible foundation by combining API-based access, containerized execution, and cloud deployment through AWS ECS [6].

10.5 Technology Selection and Justification

Before finalizing the technology stack, we reviewed comparable tools across the key layers of the MLOps pipeline, including orchestration, experiment tracking, data and artifact versioning, object storage, model serving, and deployment.

Our goal was to select tools that were not only technically suitable, but also practical for the project timeline, team expertise, and available infrastructure.

The main decision criteria included:

- technical fit with project requirements such as reproducibility, scalability, and traceability
- compatibility with the existing two-stage pipeline
- operational complexity and maintenance effort
- support for reproducibility and auditability
- team familiarity and learning curve
- infrastructure and budget constraints

Team familiarity was also an important factor. Since we already had hands-on experience with several tools through classes and lab work, choosing familiar technologies reduced onboarding risk and improved delivery confidence.

Table 4: Technology Selection and Tool Justification for the MLOps Pipeline

Category	Selected Tool	Alternatives Considered	Why Selected for This Project
Workflow Orchestration	Apache Airflow	Prefect, Dagster	Best fit for multi-stage DAG execution, scheduling, retry policies, and dependency management across ingestion → feature engineering → Stage 1/2 training → evaluation/prediction. Strong operational maturity and team familiarity from course work
Experiment Tracking & Model Governance	MLflow	Weights & Biases (W&B), Neptune.ai	Open-source, self-hostable, low integration friction with current pipeline, and strong artifact/metric/model logging. Better aligned with reproducibility and local+cloud portability requirements
Data/Artifact Versioning	DVC	LakeFS, Git LFS	Pipeline-aware dataset/artifact versioning with reproducible stage outputs. Better fit than Git LFS for ML lineage, and lighter adoption burden than LakeFS for current project scale
Object Storage (Local/CI)	MinIO (S3-compatible)	Direct AWS S3-only, local filesystem-only	S3-compatible API enabled a consistent storage workflow across local/CI and cloud-targeted design. Easier testing and reproducibility than S3-only; more traceable and scalable than filesystem-only
Cloud Hosting Path	AWS-based deployment path	Local-only execution, non-cloud VM-only path	AWS Academy credits made cloud deployment feasible. Supports realistic hosting, accessibility, and production-oriented architecture validation beyond local development
Model Serving API	FastAPI	Flask, BentoML	Typed schemas, strong validation, auto-generated OpenAPI docs, and fast integration with stage-based inference logic. Best balance of developer productivity and API robustness
Containerization	Docker / Docker Compose	Podman-only, non-containerized setup	Consistent runtime across development, CI, and deployment. Essential for multi-service stack (Airflow, MLflow, MinIO, API) and reproducible operations.
CI/CD	GitLab CI/CD	GitHub Actions, Jenkins	Already aligned with repository and team workflow; enables automated checks, test gating, and repeatable build/deploy steps with low operational overhead.

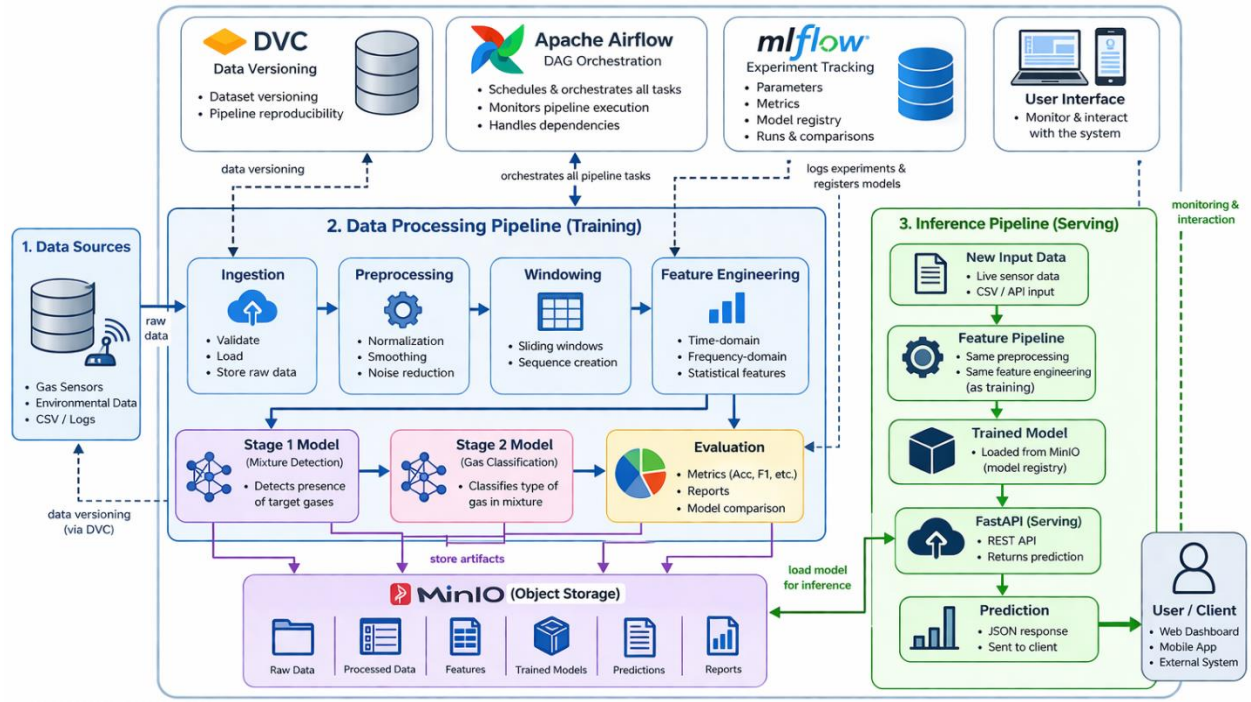


Figure 9 :End-to-End MLOps Architecture for the Gas Sensing System: Data Processing, Model Training, and Inference Pipelines with Integrated Orchestration, Versioning, and Experiment Tracking.

10.6 Pipeline Data Flow and Stages

As illustrated in Figure 10, the pipeline follows a structured flow from data ingestion to prediction serving, with each stage producing artifacts consumed by subsequent stages.

The pipeline follows an artifact-driven architecture, where data is passed between stages through persisted files rather than in-memory communication. Each stage produces structured artifacts that are stored on disk and consumed by subsequent stages.

These artifacts include Parquet files for datasets, JSON files for summaries and evaluation reports, and serialized model files for trained models. This design ensures loose coupling between stages, improves reproducibility, and enables independent execution and debugging of each pipeline component.

The use of DVC further reinforces this approach by explicitly defining stage inputs and outputs, allowing versioning and traceability of all intermediate artifacts throughout the pipeline.

All pipeline stages communicate via file-based artifacts. No in-memory data transfer is used across stage boundaries. Each stage reads its required inputs from disk and writes outputs back to disk, ensuring clear separation between pipeline components.

The ML pipeline consists of seven orchestrated stages, each representing a distinct phase in the gas sensor analysis workflow.

Stage 1: Data Ingestion

This stage ingests raw sensor data and performs initial preprocessing, translating raw sensor time-series data into standardized formats with temporal alignment and run-aware grouping (Literature Review, Section 2, Data Preprocessing) [7]. The stage produces three key artifacts:

- Preprocessed sensor measurements with group identifiers
- Experiment metadata and run groupings
- Ingestion and preprocessing statistics

Stage 2: Feature Engineering

This stage applies time-series windowing and feature extraction to create meaningful representations of sensor behavior (Literature Review, Section 3.1, Windowing and Section 3.2, Feature Engineering) [7]. The stage splits the time-series into fixed-length windows and generates both temporal and statistical features. The stage produces five outputs:

- Windowed sensor measurements
- Training set windows
- Test set windows
- Extracted feature vectors with labels
- Feature extraction statistics

Stage 3: Train Stage-1 Model

This stage trains the first-stage classifier for mixture detection by evaluating multiple candidate models under stratified group-aware cross-validation (preserving run groupings). The best performer—Random Forest with 96.1% accuracy and F1=0.96 (Literature Review, Section 4, ML Models; Notebook 03, Section 4.5)—is selected and serialized [7,10]. The stage produces:

- Serialized Stage 1 model (Random Forest)
- Cross-validated metrics for all candidate models
- Detailed Stage 1 cross-validation results with mean accuracy/F1 ± standard deviation
- Stage 1 predictions on training data

Stage 4: Prepare Stage-2 Subset

This stage uses the Stage 1 model to identify positive (mixture-containing) windows and extracts single-gas samples for Stage 2 training. The trained Stage 1 classifier filters features corresponding only to single-gas scenarios. The stage produces:

- Feature vectors for single-gas samples
- Size and distribution statistics of Stage 2 training data

Stage 5: Train Stage-2 Model

This stage trains the second-stage classifier for gas probability estimation among single-gas samples. The best-tuned model—Logistic Regression with hyperparameters C=0.01 and no class weighting, achieving 79% accuracy and F1=0.62 (Literature Review, Section 4, ML Models; Notebook 04, Section 6)—is selected [7,11]. This stage handles the feature-level overlap challenge between Toluene and 2-butanone that constrains Stage 2 performance. The stage produces:

- Serialized Stage-2 model (Tuned Logistic Regression)
- Baseline metrics for all Stage-2 candidate models
- Hyperparameter tuning experiments and performance comparisons
- Detailed Stage-2 cross-validation results
- Stage-2 predictions on training data

Stage 6: Evaluate

This stage computes comprehensive evaluation metrics for both Stage 1 and Stage 2 models on held-out test data (Literature Review, Section 5, Evaluation Strategy) [7]. Evaluation includes confusion matrices, per-class precision/recall, and F1 scores. The stage produces:

- Stage-1 evaluation metrics
- Stage-2 evaluation metrics

Stage 7: Batch Prediction

This stage applies both trained models in cascade to generate gas identification predictions on the full dataset. Stage 1 predictions identify mixture-containing windows, and Stage 2 identifies the gas type among single-gas samples. The stage produces:

- Final end-to-end gas type predictions with confidence scores
- Summary statistics on batch prediction execution

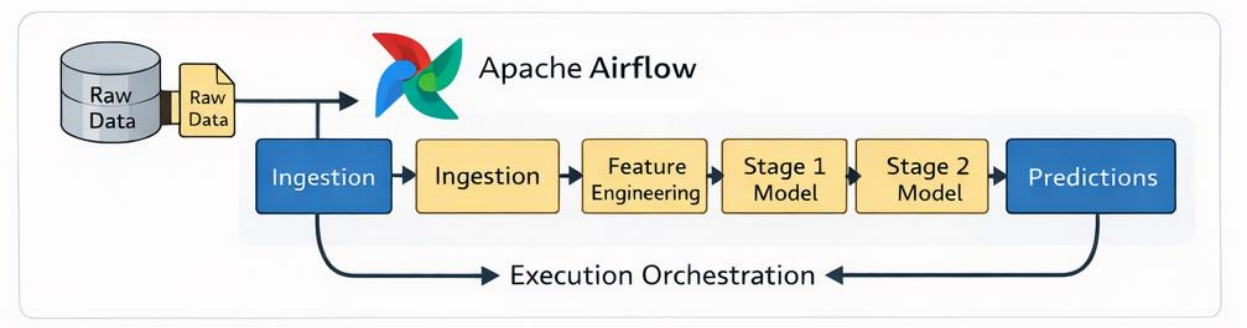


Figure 10: End-to-End Two-Stage Machine Learning Pipeline with Airflow Orchestration

10.7 Design Decisions and Trade-offs

Several key design decisions were made during the development of the system, each involving specific trade-offs.

First, a feature-based modeling approach was selected instead of directly using raw time-series data. While deep learning models could potentially capture more complex temporal dependencies, they require larger datasets and higher computational resources. Given the limited dataset size and high variability, feature-based models provide a more stable and interpretable solution. Exploration of neural networks in the project showed convergence issues and unstable performance, reinforcing this choice.

Second, the pipeline is designed as an artifact-driven system rather than relying on in-memory data transfer between stages. This introduces additional storage overhead, but significantly improves reproducibility, traceability, and modularity, allowing each stage to be executed independently. DVC is used to version and track these artifacts, creating an immutable record of data lineage across pipeline stages.

Third, an S3-compatible object storage layer is used instead of relying only on the local file system. Although this adds setup and integration complexity, it provides centralized and scalable artifact management and better aligns the system with production-oriented MLOps practices. In the current implementation, MinIO is used for local development and CI, while native AWS S3 is used in production. DVC uses this storage layer for dataset and pipeline versioning, and MLflow stores experiment artifacts through the same object-storage-backed workflow.

Fourth, group-based cross-validation is used instead of random splitting. While this results in lower performance metrics, it prevents data leakage between runs and provides a more realistic evaluation of model generalization. The project implements this via StratifiedGroupKFold to respect the temporal and run-based structure of the data.

Finally, a two-stage modeling approach is adopted instead of a direct multi-class classification model. This increases system complexity but better reflects the hierarchical structure of the problem by separating mixture detection from single-gas identification. As detailed in notebooks 03 and 04, this enables focused training on single-gas subsets for Stage 2, improves interpretability, and supports task-specific optimization. In the current implementation, Random Forest is selected as the final Stage-1 model based on superior run-aware cross-validation performance, while tuned Logistic Regression is selected as the final Stage-2 model for its best F1 under group-aware tuning and more interpretable probability estimates. (Literature Review, Section 4, Machine Learning Models for Gas Sensor Classification), (Notebook 03, Section 4.5 Model Comparison; Section 6 Key Findings and Next Step), (Notebook 04, Section 5.1 Logistic Regression Tuning; Section 6 Final Model Selection) [7,10,11]

11. Deployment and Implementation

The proposed system is implemented as an integrated machine learning pipeline that connects all major workflow stages, from data ingestion through model serving. The implementation is designed to ensure reproducibility, modularity, and production-readiness through containerization, orchestration, and artifact-driven execution.

Containerization and Multi-Service Architecture:

To ensure consistent execution across environments, the system is containerized using Docker [5]. Multiple services are deployed as separate containers, including Apache Airflow for orchestration, MLflow for experiment tracking and artifact management, MinIO for local object storage (with AWS S3 used in production), and a FastAPI-based service for model predictions. This modular architecture enables independent scaling and updates of individual components while maintaining clear service boundaries.

Pipeline Orchestration and Execution:

Pipeline execution is managed using Apache Airflow, where each workflow stage is defined as an independent task within a Directed Acyclic Graph (DAG). The DAG explicitly defines task dependencies, ensuring controlled execution from data ingestion to evaluation and prediction. Retry mechanisms and execution rules are configured to improve robustness and minimize failure impact.

Reproducibility and Experiment Tracking:

Reproducibility is enforced through containerization, DVC-based data and pipeline versioning, and MLflow-based experiment tracking. DVC defines pipeline stages and tracks intermediate artifacts including processed datasets, extracted features, trained models, and evaluation outputs. MLflow records model parameters, metrics, and artifacts to support experiment comparison and full traceability across runs. The artifact-driven design ensures that all outputs are persisted and versioned, enabling auditable reruns and stage-level inspection.

Model Serving and Cloud Deployment:

The system exposes a prediction interface through a FastAPI application, providing endpoints for health monitoring, model information, and prediction requests. This enables structured interaction with trained models. In addition to local development support, the system is deployed in an AWS environment via GitLab CI, where the API image is built, pushed to ECR, and the ECS

service is updated. This cloud-hosted setup demonstrates practical deployment-readiness and accessibility beyond a local prototype.

Overall, the implementation realizes a production-capable MLOps architecture that combines modular pipeline execution, containerized services, automated CI/CD workflows, artifact-driven reproducibility, experiment tracking, and cloud-hosted deployment on AWS ECS. The architecture scales from local development through CI testing to production deployment in a unified and repeatable manner.

12. Discussion

The results of the modeling and evaluation phases provide important insights into both the capabilities and limitations of the proposed system. While the achieved performance levels are moderate, they reflect the inherent complexity of the problem rather than shortcomings in the modeling approach.

One of the most significant observations is the impact of variability across experimental runs. Sensor responses exhibit considerable differences even under similar conditions, making it difficult for models to learn stable and generalizable patterns. This variability introduces a form of distribution shift, where the data seen during training does not fully represent the data encountered during testing. As a result, model performance is constrained by the underlying data characteristics rather than the choice of algorithm.

Another key factor influencing performance is the temporal nature of the data. Relevant information is distributed across the evolution of the signal, and simple representations may fail to fully capture complex dynamic behavior. Although the feature engineering process incorporates temporal and structural features, certain patterns may still remain difficult to distinguish, particularly in cases where different gases produce similar response profiles.

The evaluation strategy also plays a critical role in shaping the observed results. By using group-based cross-validation, the system avoids data leakage and provides a realistic estimate of performance [4]. However, this approach inherently leads to lower accuracy compared to random splitting, as it enforces a more challenging generalization scenario. This highlights the importance of choosing evaluation methods that reflect real-world conditions rather than optimizing for artificially high-performance metrics.

The comparison of different models indicates that, for Stage 1, Random Forest performs best on the current engineered feature space under run-aware validation, with stronger mean F1 and stability than the linear baseline. Random Forest achieves 96.1% F1, significantly surpassing the Logistic Regression baseline (91% F1) and proving to be the most robust model under run-aware validation (Notebook 03, Section 4.5) [10]. For Stage 2, tuned Logistic Regression performs best after hyperparameter optimization, with 79% accuracy and F1=0.62, reflecting the more difficult discrimination problem between similar single-gas responses (Literature Review, Section 4, Machine Learning Models for Gas Sensor Classification; Notebook 04, Section 6 Final Model Selection) [7,11]. This suggests that feature quality and leakage-safe evaluation have a larger impact than model complexity alone in the current data regime.

Experiments with dimensionality reduction and neural networks further reinforce this conclusion. The limited impact of these techniques suggests that performance improvements are unlikely to be achieved solely through increasing model complexity. Instead, future improvements may require enhanced feature representations, better data quality, or domain-specific modeling approaches.

From a system perspective, the integration of MLOps components demonstrates that the pipeline is robust, reproducible, and extensible. This provides a strong foundation for further development, even if current predictive performance is limited. The ability to systematically track experiments and manage data ensures that future iterations can build upon the current system in a structured manner.

Overall, the discussion highlights that the primary challenge of this project lies in the data rather than the modeling approach. The system successfully captures meaningful patterns and provides a realistic assessment of performance, while also identifying key areas for improvement in future work.

13. Quality Control and Reproducibility

Quality control and reproducibility are achieved through a combination of structured evaluation, versioned pipelines, and systematic tracking of experiments and artifacts.

A foundational aspect of quality assurance is the use of group-based cross-validation [3]. By stratifying splits across experimental runs rather than individual samples, the system prevents data leakage and ensures that evaluation metrics reflect realistic generalization performance. This approach is implemented using StratifiedGroupKFold with run_id as grouping key and 3-fold validation; metrics are reported as mean and standard deviation across folds to reflect generalization on unseen runs. (Literature Review, Section 5, Evaluation Strategy for Gas Sensor Classification), (Notebook 03 Section 3.2, Notebook 04 Section 3.2) [7,10,11]

Reproducibility is enforced through a three-layered strategy. First, DVC tracks data and pipeline versioning, with stages, dependencies, and outputs explicitly defined in dvc.yaml. This enables auditability of intermediate artifacts such as processed datasets, extracted features, trained models, and evaluation results. Second, MLflow records model parameters, configurations, metrics, and artifacts for each experiment run, supporting comparison and traceability across runs. Conditional configuration allows MLflow logging to be enabled or disabled based on runtime context. Third, containerization via Docker Compose standardizes the execution environment across development, CI, and deployment contexts, with services defined for Airflow, MLflow, MinIO (local development), and the API.

Pipeline orchestration is managed by Apache Airflow, which enforces structured execution through explicitly defined task dependencies and DAG-based workflow control. This ensures controlled progression from data ingestion through evaluation to batch prediction, improving both robustness and observability.

In production, deployment to AWS-hosted environments further strengthens reproducibility by reducing dependency on local infrastructure and ensuring consistency across execution contexts. The combination of versioned artifacts, tracked experiments, containerized services, and orchestrated execution creates a comprehensive reproducibility framework that aligns with modern MLOps practices [2].

14. Continuous Integration and Continuous Deployment (CI/CD)

The project implements continuous integration (CI) using GitLab CI, a pipeline orchestration tool that automatically validates and tests all repository changes. The CI pipeline is designed to ensure code reliability, correctness, and pipeline reproducibility.

Automated Testing and Quality Assurance

The CI workflow includes automated validation at multiple layers: unit testing validates core logic, integration testing verifies pipeline behavior and data flows, and API testing confirms model-serving functionality. These tests execute automatically on each push and merge request, preventing regressions and ensuring consistency across system components throughout development.

Reproducibility and Infrastructure Validation

Beyond traditional testing, the CI pipeline incorporates reproducibility validation. DVC pull and repro commands verify that data processing and model training can be reproduced from versioned artifacts. Infrastructure integration testing performs end-to-end validation by running containerized services to verify integration between MinIO, MLflow, and DVC. These resource-intensive jobs are configured as manual triggers for push and merge request events, while automatic execution is enabled for scheduled pipeline runs, balancing comprehensive validation with practical resource management.

Model Publishing and Deployment Automation

The pipeline includes automated model publishing jobs that build model bundles and push them to S3 object storage. An automated ECS deployment job consumes published model versions and registers updated task definitions with AWS ECS, deploying the API with current model versions. This represents significant progress toward full CD automation for the serving layer.

Current CI/CD Maturity

The system is classified as a CI-enabled pipeline with partial CD automation. Automated testing and reproducibility validation are comprehensive, and model serving deployment to AWS ECS is automated. However, deployment of other services such as Airflow and MLflow orchestration services still requires manual configuration. The existing containerization and cloud infrastructure provide a strong foundation for extending full CI/CD automation across all system components as operational requirements evolve.

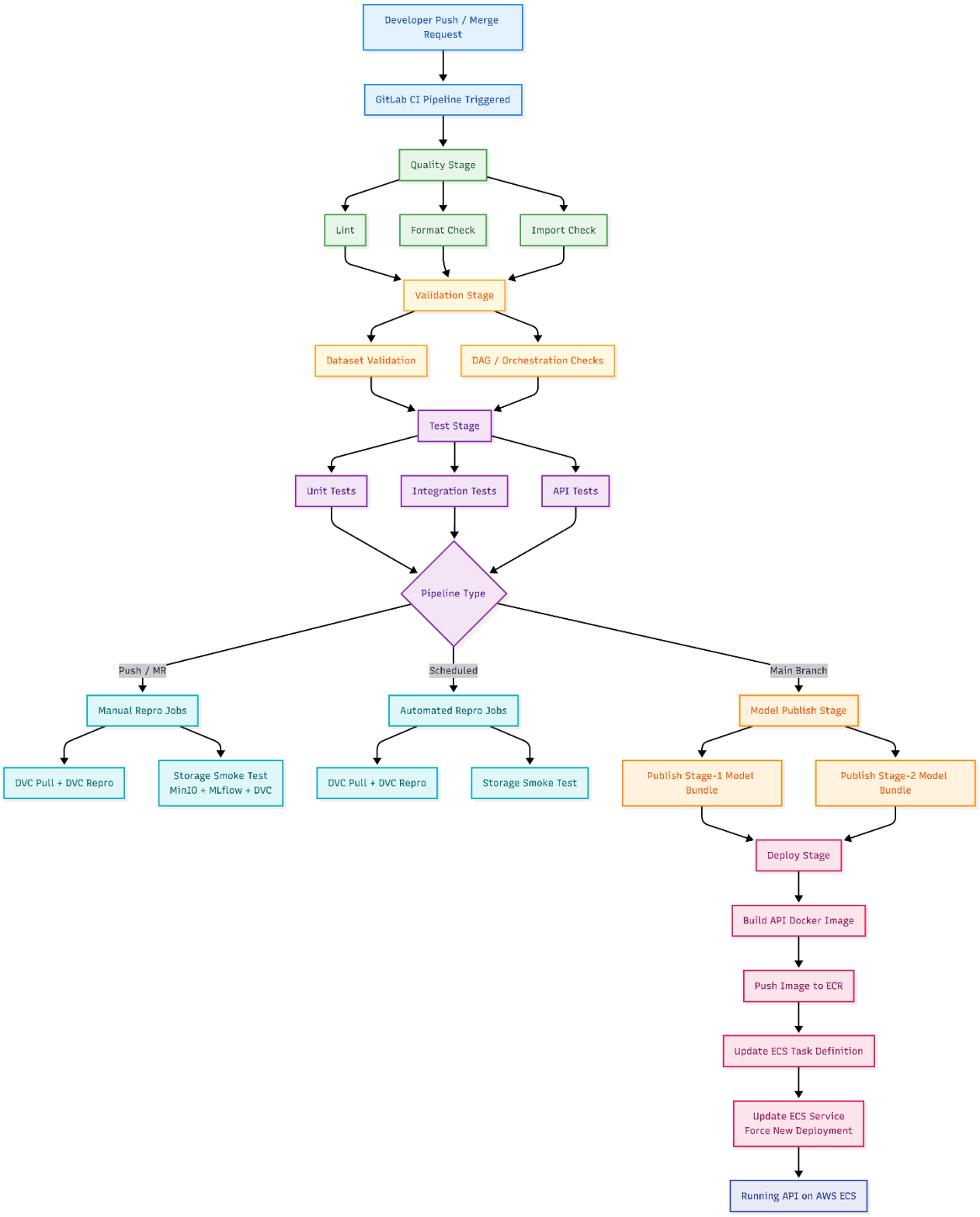


Figure 11: GitLab CI/CD Workflow for Testing, Model Publishing, and AWS Deployment

15. Limitations and Future Work

Despite the strengths of the proposed system, several limitations remain that affect predictive performance, operational scalability, and long-term reliability. These limitations also define a clear roadmap for future improvements.

A primary limitation is data quality and variability. Sensor responses show substantial run-to-run variation, even under similar experimental conditions, which reduces model generalization. In addition, the dataset size is relatively limited, constraining statistical robustness and reducing the feasibility of training more expressive models with stable performance.

A second limitation is the feature-based modeling strategy. Although engineered features capture important statistical and temporal characteristics, they may not fully preserve complex signal dynamics present in raw time-series data. As a result, subtle but informative patterns can be attenuated during feature extraction, potentially limiting classification performance.

Data augmentation was also assessed during technical discussions with the client as a potential mitigation for limited data volume. However, given the physical nature and structure of the sensing signals, augmentation was not considered a reliable improvement path at this stage. Generic augmentation methods can distort physically meaningful signal behavior and introduce synthetic patterns that are not representative of real sensor responses. This can inflate training performance without delivering robust gains in real-world generalization. For this reason, the project prioritized improving data quality, run diversity, and validation rigor over synthetic sample generation.

Another important limitation is the absence of explicit drift handling. Sensor drift and long-term distribution shift are common in gas sensing systems, yet the current pipeline does not include dedicated drift detection, adaptation, or recalibration mechanisms. This may lead to performance degradation over time in real deployment environments.

From a systems perspective, the pipeline is currently batch-oriented. Execution is manual or schedule-driven rather than event-driven, which limits applicability in streaming and low-latency inference scenarios. Operational monitoring is also incomplete: while experiment tracking is supported, production-grade post-deployment monitoring, alerting, and automatic retraining triggers are not fully implemented.

From a DevOps perspective, continuous integration is well established, but end-to-end continuous deployment remains partially automated. Some release and environment-

management activities still require manual intervention, increasing operational overhead and reducing deployment velocity.

Future work should focus on five directions. First, expand and diversify the dataset to improve robustness across runs and conditions. Second, evaluate advanced sequence-learning methods once data scale and quality are sufficient. Third, integrate drift detection and adaptive update strategies for long-term reliability. Fourth, extend the architecture toward streaming inference for near-real-time use cases. Fifth, implement production monitoring, automated retraining policies, and fuller deployment automation to complete the transition to a mature, cloud-ready MLOps platform.

Overall, the current system provides a strong and extensible foundation for gas-sensing machine learning, but addressing these limitations is essential to achieve higher performance, sustained reliability, and production-grade operational maturity.

16. Conclusion

This project presented the design and implementation of an end-to-end machine learning pipeline for gas-sensing applications, with a strong emphasis on reproducibility, modularity, and deployment-oriented system design.

From a data perspective, the work highlighted the core challenges of sensor time-series modeling, including run-to-run variability, non-stationary behavior, and limited data volume. These constraints significantly influenced model behavior and reinforced the need for robust preprocessing, feature engineering, and leakage-aware evaluation.

From a modeling perspective, a two-stage classification strategy was adopted to address the hierarchical structure of the prediction problem. Multiple models were evaluated using group-based cross-validation to obtain realistic generalization estimates. Results indicated stage-dependent behavior: a non-linear model, Random Forest, was best for Stage 1 with 96.1% accuracy and F1=0.96, while a tuned linear model, Logistic Regression, was best for Stage 2 with 79% accuracy and F1=0.62; in both stages, data characteristics remained the dominant limiting factor. (Notebook 03, Section 4.5; Section 6), (Notebook 04, Section 6 Final Model Selection) [10,11]

From a systems and MLOps perspective, the project integrated orchestration, versioning, tracking, serving, and deployment into a unified workflow: Airflow for pipeline orchestration, DVC for data and artifact versioning, MLflow for experiment tracking, Docker for environment consistency, and FastAPI for model serving.

The solution also demonstrated cloud applicability through AWS-hosted deployment and GitLab-based CI/CD workflows. Automated validation and testing are in place, and deployment automation for the serving layer has been established, moving the system beyond a local prototype toward practical operational use.

Although capabilities such as real-time streaming inference, comprehensive post-deployment monitoring, and fully automated retraining are still future targets, the current implementation provides a solid and extensible foundation for production-grade evolution.

Overall, this work shows that effective machine learning systems depend not only on model accuracy, but also on reproducible pipelines, robust engineering, and deployable architecture. By combining data-driven modeling with structured MLOps and cloud deployment, the project delivers a practical and scalable foundation for gas-sensing intelligence, with clear directions for future enhancement.

References

[1] IBM Corporation. (2021). IBM SPSS Modeler CRISP-DM Guide (Version 18.3.0). IBM Support.

[2] Treveil, M., et al. (2020). Introducing MLOps. O'Reilly Media.

[3] Roberts, D. R., et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. Ecography, 40(8), 913–929.

[4] Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4), 1–21.

[5] Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. Linux Journal, 2014(239).

[6] Amazon Web Services. (2023). Overview of Amazon Web Services. AWS Whitepaper. https://aws.amazon.com/whitepapers/

[7] Project Documentation. Literature Review for OBSeRVeD CMOS VOC Pipeline. Project report.

[8] Project Documentation. Notebook 01: Business and Data Understanding for Time-Series Sensor Data.

[9] Project Documentation. Notebook 02: Feature Engineering for Time-Series Data.

[10] Project Documentation. Notebook 03: Stage-1 Model Training - Mixture Detection.

[11] Project Documentation. Notebook 04: Stage-2 Model Training - Gas Probability Estimation.